

File Search Program Project Report

Ayşe Feyza SERBEST

March 13, 2026

1 Problem Definition

The goal of this project was to implement a command-line tool similar to the Unix `find` utility. The program searches files inside a given directory according to different filtering options such as file name, type, or path.

The main challenge of the project was understanding how file system traversal works in Unix-like systems and how low-level system calls can be used to inspect files and directories. Instead of relying on high-level libraries, the program had to interact directly with the operating system through system calls.

Another challenge was designing a command-line interface that correctly interprets user arguments and applies the requested filters while traversing the directory tree.

2 Solution Approach

To solve this problem, I implemented a recursive directory traversal mechanism that explores directories and processes files according to the given options.

Instead of using the standard `getopt` function for argument parsing, I decided to implement my own argument parser. This helped me better understand how command-line arguments are structured and how they can be processed manually.

The program works as follows:

1. Parse command-line arguments.
2. Determine which filters should be applied.
3. Traverse the target directory.
4. Inspect each file using system calls.
5. Apply filters and print matching results.

This approach allowed the program to mimic the behavior of a simplified `find` command while giving me deeper control over argument handling and file inspection.

3 Technical Knowledge Gained

During the development of this project, I gained several important technical skills.

First, I learned how system call-based functions work internally. Functions that interact with the file system rely on kernel-level operations, and understanding their behavior helped me better grasp how operating systems manage files and directories.

While implementing these parts of the project, I frequently consulted the Linux manual pages (`man` pages) to understand the behavior, parameters, and return values of different system calls and library functions. This was particularly useful when working with file-related functions and signal handling.

Second, implementing my own argument parsing mechanism improved my understanding of command-line program design. Instead of using `getopt`, manually parsing arguments forced me to carefully handle different argument combinations and edge cases.

Finally, I improved my understanding of signal handling in Unix systems, particularly the behavior of signals such as `SIGINT`. Studying the documentation of signal-related functions through the Linux manual pages helped me better understand how signal handlers are registered and how signals interrupt program execution.

4 Challenges and Failures

One of the most difficult parts of the project was handling signals correctly.

The program executes very quickly while scanning directories. Because of this speed, the `SIGINT` signal (triggered by pressing `CTRL+C`) often arrived either before the program started processing or after it had already finished. As a result, the signal handler did not always behave as expected.

This issue made debugging difficult because the signal handling code was correct, but the program simply terminated too quickly for the signal to be processed during execution.

5 How the Problem Was Resolved

To solve this issue, I modified the function responsible for searching inside the target directory.

Inside the `while` loop that iterates through directory entries, I introduced a small delay using the `usleep` function. This slight pause slowed down the execution enough to allow the `SIGINT` signal to be received and handled properly while the program was running.

This change allowed the signal handler to work as intended and made it possible to safely interrupt the program during execution.

6 Conclusion

This project helped me develop a deeper understanding of Unix system programming. By implementing directory traversal, manual argument parsing, and signal handling, I gained practical experience with low-level operating system concepts.

Although I faced difficulties—especially with signal timing issues—these challenges ultimately helped me better understand how programs interact with the operating system and how timing can affect signal handling behavior.

Overall, the project strengthened my skills in systems programming and improved my ability to debug problems related to operating system behavior.